

Lab 04 - Masking, Inpainting, Outpainting and Background Swap

Course: Generative AI for Image and Video Creation (TGS-2020505925) | Version v11.0 | 6 September 2026 Topic 02: AI Image Generation, Editing and Visual Enhancement Outcome: ELO2 | In-class time: 15 minutes of scheduled practical time

Competency	Statement
Ability A2	Apply the principles of processing, filtering and analysis methods for video data
Knowledge K3	Methods to represent image and video data
Knowledge K4	Image and video processing, filtering and transformation methods

Scenario

A product shot has a stray price tag in frame, the wrong background, and it is square when the campaign needs a 16:9 hero. Masks help specify all three edits, but output quality also depends on the repair method. Build the masks yourself in OpenCV, run the deterministic classical repair to see the mechanism, measure what changed inside and outside the mask, then write the generative edit specification that a model would execute. Classroom core (15 minutes): run the three bundled mask jobs, inspect preservation measurements and write the edit/preserve specification. Building the implementation from scratch and additional provider trials are extensions.

What you will produce

Three masks (out/mask_tag.png, out/mask_bg.png, out/mask_outpaint.png), the classical inpaint results, out/mask-audit.csv proving the unmasked region was preserved, and out/edit-spec.json specifying the generative equivalent.

Tools: Python 3 | opencv-python | NumPy | offline with preinstalled dependencies; no paid service

Environment

Install these once, before class if you can - the lab itself needs no network access after the dependencies are present:

```
python3 -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\Activate.ps1
pip install --upgrade pip
pip install "opencv-python>=4.10" numpy
python3 -c "import cv2, numpy; print(cv2.__version__, numpy.__version__)"
```

Required: python3, opencv-python, numpy.

API currency. Everything taught here runs on the base opencv-python wheel. Two things that appear in older tutorials are *not* in that wheel and are deliberately avoided: cv2.TrackerCSRT_create / cv2.TrackerKCF_create and the cv2.quality SSIM module, both of which live in opencv-contrib-python. Where SSIM is needed the course ships its own NumPy implementation.

Workflow

1. Build a binary mask for the unwanted object
2. Inpaint with Telea and Navier-Stokes
3. Audit change inside vs outside the mask
4. Build the background and outpaint canvases
5. Write the generative edit specification

Files in this folder

Path	What it is
reference/nq-jacket-square.png	Synthetic 1024x1024 product shot with a subject, a plain backdrop and a stray price-tag rectangle (deterministically rendered with OpenCV - SIMULATED)
reference/nq-backdrop-harbour.png	Synthetic 1820x1024 replacement backdrop (SIMULATED)
data/edit-jobs.json	Three edit jobs with their target regions, prompts and acceptance thresholds
data/mask-conventions.md	The binary-mask convention reference: white = editable, black = preserved
mask_edit.py	Runnable script - see the steps below
PROMPTS.md / PROMPTS.pdf	The reusable prompt and specification pack
out/	Everything you produce (created on first run)

Provenance. Every image and clip in reference/ is either a SIMULATED classroom asset rendered deterministically with OpenCV, an AI-generated reference copied unchanged with a PROVENANCE.md beside it, or a real prerecorded sequence with its source and SHA-256 recorded. Nothing in this folder may be relabelled: a rendered animation is never presented as model output, and a simulated stand-in is never presented as a photograph.

Steps

1. Start in this lab folder with prepared OpenCV/NumPy dependencies. Create out/. For Python fragments use one notebook or REPL: `import cv2, numpy as np; from pathlib import Path; Path('out').mkdir(exist_ok=True); from mask_edit import read, background_mask; img=read('reference/nq-jacket-square.png')`. The full script performs all jobs; fragment demonstrations share this context.
2. Read data/mask-conventions.md. The convention used in this lab and by cv2.inpaint is: a single-channel 8-bit image where WHITE (255) marks the editable pixels and BLACK (0) marks preserved pixels. Check provider-specific semantics before using an API, and verify output preservation. Getting this inverted can cause masked-edit failure.
3. Load reference/nq-jacket-square.png and confirm it is 1024x1024x3 uint8. Locate the price tag visually and note its approximate bounding box.

```
python3 -c "import cv2;i=cv2.imread('reference/nq-jacket-square.png');print(i.shape)"
```

4. Build mask 1 - the price tag - by COLOUR rather than by hand-typed coordinates. The tag is drawn in a saturated yellow; threshold in HSV so the mask survives if the product moves. Save it as out/mask_tag.png.

```
hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
mask = cv2.inRange(hsv, (20, 120, 120), (35, 255, 255)) # H 20-35 = yellow
cv2.imwrite('out/mask_tag.png', mask)
```

5. Clean the mask with morphology: `cv2.MORPH_CLOSE` with a 5x5 kernel fills pinholes, then a single `cv2.dilate` grows the mask a few pixels past the object edge. That dilation is what lets the fill blend - a mask cut exactly on the boundary leaves a halo.

```
k = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5, 5))
mask = cv2.morphologyEx(mask, cv2.MORPH_CLOSE, k)
mask = cv2.dilate(mask, k, iterations=2)
cv2.imwrite('out/mask_tag.png', mask)
```

6. Report the mask coverage as a percentage of the frame. A tag mask should be a low single-digit percentage; if it is 30% your hue band is catching the product too.

```
print('coverage %.2f%%' % (100.0 * (mask > 0).mean()))
```

7. Run classical inpainting twice - `cv2.INPAINT_TELEA` (fast marching) and `cv2.INPAINT_NS` (Navier-Stokes). Both propagate surrounding texture inward; neither invents new content. That difference is exactly what separates classical repair from generative inpainting.

```
telea = cv2.inpaint(img, mask, 3, cv2.INPAINT_TELEA)
ns = cv2.inpaint(img, mask, 3, cv2.INPAINT_NS)
```

8. Vary the `inpaintRadius` argument across 1, 3 and 9 and look at the three results. Larger radius pulls texture from further away: smoother on flat backdrops, smeared across a textured one. Record your chosen radius and review its artefacts.

```
for radius in (1,3,9):
    cv2.imwrite(f'out/telea_radius_{radius}.png', cv2.inpaint(img, mask, radius, cv2.
INPAINT_TELEA))
```

9. Audit the edit. Compute the mean absolute difference between the original and the result INSIDE the mask and OUTSIDE the mask. Outside must be effectively zero - that is the numeric proof that the edit was contained.

```
python3 mask_edit.py --image reference/nq-jacket-square.png --jobs data/edit-jobs.json --
out out
```

10. Build mask 2 - the background - as the complement of the subject. Segment the subject with Otsu on the saturation channel, take the largest connected component, fill it, then invert. Save as `out/mask_bg.png`.

```
s = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)[: , : , 1]
_, fg = cv2.threshold(s, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)
bg, fg = background_mask(img) # largest component, filled holes
cv2.imwrite('out/mask_bg.png', bg)
```

11. Composite the supplied replacement backdrop through mask 2 to see the mechanical background swap. Note where it fails: the hard matte edge and any colour spill the original lighting left on the subject. A generative background swap may improve the boundary by re-rendering it but may introduce new errors; alpha matting and colour correction can also improve composites.

```
bg_img = cv2.resize(cv2.imread('reference/nq-backdrop-harbour.png'), (img.shape[1], img.
shape[0]))
swapped = np.where(bg[:, :, None] > 0, bg_img, img)
```

12. Build mask 3 - the outpaint canvas. Pad the 1024x1024 square onto a 1820x1024 canvas (approximately 16:9, 1.77734:1) with the original centred, then create a mask that is BLACK over the original pixels and WHITE over the new empty side panels. This is the classroom canvas contract; each outpainting API may require different inputs.

```

canvas = np.zeros((1024, 1820, 3), np.uint8)
x0 = (1820 - 1024) // 2
canvas[:, x0:x0 + 1024] = img
om = np.full((1024, 1820), 255, np.uint8)
om[:, x0:x0 + 1024] = 0
cv2.imwrite('out/mask_outpaint.png', om)

```

13. Confirm the outpaint mask geometry numerically: the white region must be exactly 2 x 398 x 1024 pixels and the black region exactly 1024 x 1024. If the numbers are off by one, your centring arithmetic is wrong and the seam will show.

```

print('white', int((om > 0).sum()), 'expected', 2 * 398 * 1024)

```

14. Open data/edit-jobs.json and read the three acceptance thresholds. Job 1 (tag removal) requires outside-mask mean absolute difference < 0.5 intensity levels on the 0-255 scale; job 2 (background) requires the subject's HSV histogram to stay within a Bhattacharyya distance of 0.15 of the original; job 3 (outpaint) requires the central 1024 columns to be bit-identical to the source.

15. Check every job against its threshold in out/mask-audit.csv and mark each PASS or FAIL. The CSV reports geometry/pixel checks, not semantic quality. Inspect the segmented silhouette against the actual jacket; an incorrect matte can still pass preservation metrics. Job 3 is only a blank canvas, not a generated outpaint. A failure requires checking masks, source alignment, encoding and algorithm/model behaviour; it does not identify one cause.

16. Write out/edit-spec.json - the generative equivalent of what you just did classically. For each job record: the mask file, the mask semantics (which colour is editable), the edit mode (inpaint-removal, background-swap, outpaint), the edit prompt, the preserve list, and the acceptance metric with its threshold.

17. Add the mask-free alternative to the spec for job 1 only, as a second entry: the same instruction expressed as a text-only edit with no mask supplied. Note in the spec the trade-off - mask-free may be faster to author; either method needs review before commercial use because a generative mask is not a universal pixel-preservation guarantee.

18. Save the three masks, the inpaint results, out/mask-audit.csv and out/edit-spec.json, then complete the evidence checklist.

Verify

out/mask-audit.csv shows job 1 with an outside-mask mean absolute difference below 0.5 intensity levels on the 0-255 scale and a clearly non-zero inside-mask difference, the outpaint white-pixel count equals 815,104, and out/edit-spec.json parses with a mask file, an edit mode, a preserve list and a numeric acceptance threshold for all three jobs.

Expected outputs

- out/mask_tag.png - single-channel, coverage in low single-digit percent.
- out/mask_bg.png - subject black, background white.
- out/mask_outpaint.png - 1024x1820, exactly 815,104 white pixels.
- out/inpaint_telea.png and out/inpaint_ns.png with the tag removed.
- out/mask-audit.csv - inside/outside mean absolute difference per job with PASS/FAIL.
- out/edit-spec.json - three jobs plus one mask-free alternative entry.

Evidence checklist

Submit all of the following. Each item is something an assessor can look at.

[] The three mask images, with the coverage percentage of each.

- [] The inside-mask and outside-mask mean absolute differences for job 1.
- [] The inpaintRadius you chose (1, 3 or 9) and one sentence of justification.
- [] The outpaint white-pixel count matching 815,104.
- [] out/edit-spec.json showing the mask semantics stated explicitly for every job.
- [] One sentence on where the mechanical background composite visibly fails and why a generative swap might help or fail.

Troubleshooting

Symptom	What to do
The inpainted region is a grey blob	Possible causes include a wrong mask, insufficient surrounding texture or an unsuitable radius. cv2.inpaint repairs where the mask is non-zero. Print mask.mean(): a tag mask should be a small number, not near 255.
cv2.inpaint raises an assertion about the mask type	The mask must be single-channel 8-bit (CV_8UC1). If you built it from a colour operation, convert with cv2.cvtColor(mask, cv2.COLOR_BGR2GRAY) first.
Outside-mask difference is not zero	cv2.inpaint only writes inside the mask, so a non-zero outside difference means you compared against the wrong source image, or you resized somewhere. Re-read the original from disk for the comparison.
The background mask captures part of the subject	Otsu on saturation splits at a single global threshold. Take only the largest connected component of the foreground and fill its holes with cv2.floodFill before inverting.
The outpaint seam is visible after compositing	That is the expected classical result and is the teaching point: this simple hard composite does not continue texture across the seam. Record it rather than trying to hide it.

References used in this lab

- [S8] Tschochohei and Schenker (2026), as above - *Ch.5 pp.68-75*.

Binary masking (white = editable, black = preserved); background swap; inpainting with semantic masks and mask dilation; outpainting / generative expand for aspect-ratio repurposing; mask-free editing and the DiffEdit source/target-prompt mask inference; precision-versus-speed trade-off between mask-based and mask-free edits.

- [S17] Mewada et al. (2026), as above - *Ch.7 pp.94-98 (Dubey, Pinheiro, Singh, Ansari and Kumar, 'Advanced Foundations and Future Trends in Generative AI for Visual Media')*.

Table 7.2 scaling strategies and their targets (trajectory distillation -> fewer sampling steps at similar FID/FVD; quantisation 8/4-bit -> reduced latency and memory; low-rank adapters -> small trainable-parameter fraction; latent diffusion -> lower bandwidth cost at high resolution; retrieval conditioning -> higher faithfulness without more parameters). Evaluation protocol: FID as a 2-Wasserstein approximation in feature space (encoder-dependent, sample-size sensitive), KID as an unbiased MMD², LPIPS as a learned perceptual distance, precision/recall for mode coverage, CLIP-based text-image faithfulness, masked-region consistency and identity preservation for edits, FVD plus optical-flow agreement for video, and watermark detectability under common edits (Table 7.3 axes and pitfalls).

- [S20] Google AI for Developers - Gemini API image generation documentation - *<https://ai.google.dev/gemini-api/docs/image-generation> (verified 6 Sep 2026)*.

Current image-generation model IDs gemini-3.1-flash-image, gemini-3.1-flash-lite-image, gemini-3-pro-image, with gemini-2.5-flash-image as the legacy model; the client.interactions.create call pattern for text-to-image, image editing and multi-turn conversational editing via previous_interaction_id; aspect ratios including 1:1, 3:2, 2:3, 3:4, 4:3, 4:5, 5:4, 9:16, 16:9 and 21:9; output sizes 1K/2K/4K by model; and the statement that all generated images include a SynthID watermark.

- [S23] OpenCV 4.13 Python package, verified locally on the delivery image - *cv2 4.13.0 (opencv-python 4.13.0.92)*.

API-currency verification for every command taught: cv2.data.harcascades ships the frontal-face and eye cascades so no weights download is required; cv2.HOGDescriptor_getDefaultPeopleDetector() is built in; SIFT_create, ORB_create, FastFeatureDetector_create, cornerHarris, goodFeaturesToTrack, matchTemplate, createBackgroundSubtractorMOG2/KNN, meanShift, CamShift, calcOpticalFlowFarneback and PSNR are all present. IMPORTANT CORRECTION carried into the labs: cv2.TrackerCSRT_create and cv2.TrackerKCF_create are NOT in the base opencv-python wheel (they require opencv-contrib-python), and the cv2.quality SSIM module is also contrib-only - so the labs use cv2.TrackerMIL_create and a NumPy SSIM implementation to stay dependency-light and runnable offline.

© 2026 Tertiary Infotech Academy Pte Ltd. UEN: 201200696W. Course material for TGS-2020505925, version v11.0.